PIT Embedding Report Aug 21

PIT-1B Embedding Extraction: Hidden State Normalization

Summary

While extracting embeddings from PIT-1B for earnings call transcripts, we observed extremely large values in certain hidden dimensions (dim 1531 reaching magnitudes of 10,000+). After investigation, we found this is **not a model deficiency** but an **extraction error**: the model applies `F.rms_norm` to the hidden state before the LM head, and we were extracting the hidden state before this normalization step. Applying `F.rms_norm` to the extracted hidden states produces well-behaved embeddings across all 11 checkpoints (2014-2024).

Root Cause

PIT is based on the modded-nanogpt architecture, which applies the final normalization as a bare functional call rather than a named module:

```
        # PIT's forward method (modeling_pit.py):
        for block in self.transformer["h"]:
            x = block(x, ...)
        x = F.rms_norm(x, (x.size(-1),))   # ← functional call, not a
named module
        logits = self.lm_head(x)
```

Standard HuggingFace models (GPT-2, LLaMA) implement this as a named module (`self.norm` or `self.ln_f`) and return post-norm hidden states via `output_hidden_states=True`. PIT's implementation does not expose this because:

1. `F.rms_norm` is a bare function call with no learnable parameters, so it does not appear in `model.named_modules()` or `model.state_dict()`
2. `output_hidden_states=True` is not implemented in the custom `PITForCausalLM` (returns `None`)
3. The only way to discover the norm is to read the `forward()` source code

This led us (and likely other researchers using hooks or `output_hidden_states`) to extract the pre-normalization hidden state, producing the large magnitude values.

Verification Across All Checkpoints

We tested the same 200 earnings call transcripts on all 11 PIT-1B checkpoints (2014-2024), extracting hidden states both with and without `F.rms_norm` applied after the last transformer block.

With `F.rms_norm` applied (correct extraction)

Checkpoint	Std	Max	Dims > 10	L2 Norm
201412	0.90	32.2	2	32.7
201511	0.90	35.0	2	34.1
201612	0.90	36.1	2	35.2
201712	0.90	36.7	1	35.9
201812	0.90	36.9	1	36.3
201912	0.90	36.8	2	36.3
202012	0.93	36.5	2	36.4
202112	0.93	35.4	2	36.3
202212	0.93	32.6	4	36.0
202312	0.89	27.8	5	35.4
202412	0.89	24.2	5	35.1

Stable across all checkpoints: std ~0.90, L2 norm ~35, at most 5 dimensions exceed abs 10. These are well-behaved embeddings suitable for downstream use.

Without `F.rms_norm` (incorrect extraction)

Checkpoint	Std	Max	Dims > 10	L2 Norm	Top Dim Mean
201412	255	2,314	1,536	1,803	1,610
201511	255	4,160	1,536	3,090	2,929
201612	255	6,308	1,536	4,805	4,664
201712	255	8,924	1,536	6,975	6,825
201812	255	10,868	1,536	8,860	8,688
201912	255	12,797	1,536	10,211	9,980
202012	291	13,687	1,536	11,368	11,025
202112	299	13,549	1,536	11,667	11,048
202212	293	12,107	1,536	11,413	10,077
202312	268	9,469	1,536	10,600	7,907
202412	255	7,479	1,536	9,981	5,989

The magnitudes grow from 2014 to ~2020 (peaking around 202012-202112) then decrease. All 1,536 dimensions exceed abs 10. These values are entirely driven by the missing normalization step.

Why This Affects Researchers

The modded-nanogpt architecture (which PIT is based on) was designed for training speed, not for the HuggingFace ecosystem. The implementation gap:

Feature	Standard HuggingFace	PIT (modded-nanogpt)
Final norm	`self.norm = RMSNorm(...)` (named module)	`F.rms_norm(x, ...)` (functional call)
`output_hidden_states`	Returns post-norm states	Not implemented (returns None)
Hook on final norm	Works (`model.norm`)	Impossible (no module to hook)

Feature	Standard HuggingFace	PIT (modded-nanogpt)
`model.named_modules()`	Includes `norm`	Does not include any final norm

Any researcher using standard extraction methods (hooks on the last block, `output_hidden_states`, or calling the backbone directly) will get pre-normalization hidden states and observe the large magnitude values.

Correct Extraction Method

```
        import torch
        import torch.nn.functional as F

        # Hook on last transformer block
        captured = {}
        def hook_fn(module, input, output):
            captured["h"] = output[0] if isinstance(output, tuple) else
output

        hook = model.transformer.h[-1].register_forward_hook(hook_fn)
        model(input_ids=input_ids, attention_mask=attention_mask)
        hook.remove()

        # Apply the same rms_norm as the model's forward method
        hidden = captured["h"]
        hidden = F.rms_norm(hidden, (hidden.size(-1),))  # ← critical step

        # Mean pool
        mask = attention_mask.unsqueeze(-1).float()
        embedding = (hidden * mask).sum(1) / mask.sum(1).clamp(min=1e-9)
```

Recommended Fix for the HuggingFace Repo

The cleanest solution is a minimal change to `modeling_pit.py` on HuggingFace to support `output_hidden_states`:

```
         def forward(self, input_ids=None, attention_mask=None,
labels=None,
        +            output_hidden_states=None, **kwargs):
        +    if output_hidden_states is None:
```

```
+        output_hidden_states = self.config.output_hidden_states

         x = self.transformer["wte"](input_ids)
         for block in self.transformer["h"]:
             x = block(x)
         x = F.rms_norm(x, (x.size(-1),))
+    hidden_states = (x,) if output_hidden_states else None
         logits = self.lm_head(x)
-    return CausalLMOutput(logits=logits)
+    return CausalLMOutput(logits=logits,
hidden_states=hidden_states)
```

With this change, embedding extraction works out of the box:

```
outputs = model(**inputs, output_hidden_states=True)
embedding = outputs.hidden_states[-1]  # post-norm, 1536 dims,
ready to use
```

No hooks, no manual normalization. The `config.output_hidden_states` defaults to `False` so there is no memory overhead for standard text generation.

Alternatively, adapting the model to a natively supported HuggingFace architecture (e.g., `LlamaForCausalLM`, which shares the same RoPE + RMSNorm design) would provide full ecosystem compatibility without any custom code.

Note on Previous Report

The previous report (Aug 20) incorrectly identified an architectural issue. The model is architecturally sound. The issue was entirely in the extraction method.